Comparative Analysis

Shipyard Agent Rebuild of Ship vs. Original FleetGraph Codebase

March 2026

1. Executive Summary

Shipyard is an autonomous coding agent built with LangGraph, FastAPI, and GPT-4o that was tasked with rebuilding the Ship application from scratch. The original Ship codebase (FleetGraph) comprises ~126,000 lines of TypeScript across a full-stack Express.js/React/PostgreSQL monorepo with real-time collaboration, a proactive AI monitoring agent, government-grade authentication, and 39 database migrations. Shipyard produced a functional rebuild ("new_ship") totaling ~5,500 lines of code (Python backend + React frontend) with JWT authentication, full CRUD operations, task management with Kanban/Gantt views, analytics dashboards, WebSocket real-time updates, and Docker containerization—all generated across 18 commits in roughly 24 hours of agent runtime. The rebuild captures the core project-management domain (projects, documents, issues, plans, tasks, users) but omits the FleetGraph intelligence agent, real-time collaborative editing, and the depth of the original's permission model and audit infrastructure. The rebuild demonstrates that an autonomous agent can stand up a working, deployable application at remarkable speed, but it also exposes clear limitations in architectural fidelity, security depth, and the ability to replicate domain-specific features like proactive anomaly detection without explicit human guidance.

2. Architectural Comparison

Technology Stack Divergence

The most striking architectural decision the agent made was a complete language and framework swap. The original Ship is a TypeScript monorepo: Express.js API, React frontend with TipTap/Yjs collaborative editing, and PostgreSQL with 39 versioned migrations. The agent rebuilt it as a Python/FastAPI backend with a vanilla React/TypeScript frontend, SQLite for persistence, and Alembic for migrations. This was not a deliberate architectural choice—it reflects the agent's default bias toward the stack it was built in (Python/FastAPI). A human developer tasked with "rebuilding Ship" would almost certainly preserve the original language to maintain compatibility and team familiarity.

Layer	Original (FleetGraph)	Rebuilt (new_ship)
Backend	Express.js + TypeScript	FastAPI + Python 3.12
Frontend	React + TipTap + Yjs (CRDT)	React 18 (vanilla, no editor)
Database	PostgreSQL (39 migrations)	SQLite (2 migrations)
Auth	Session + OIDC/SAML + PIV certs	JWT + SHA-256 password hash
Real-time	Yjs WebSocket (CRDT doc sync)	Basic WebSocket channel
Agent/AI	FleetGraph (LangGraph JS, 4-way parallel)	None
Deployment	Railway + Docker	Docker Compose

Data Model Simplification

The original uses a "unified document model" where every entity (wiki page, issue, sprint, person, standup, plan, retro) is a row in a single documents table, discriminated by document_type, with type-specific data in JSONB properties columns and relationships managed through a document_associations join table. This is a sophisticated pattern that enables generic querying, backlinks, and a flexible content model.

The agent rebuilt this as six separate normalized tables (projects, users, documents, issues, plans, tasks)—a conventional relational design. This is a choice a junior developer would make: it works correctly for CRUD but loses the original's ability to treat all content uniformly, create cross-entity associations, or support the property-bag extensibility that JSONB affords. The agent never considered the unified document pattern because it had no context about why that design existed—it only saw what entities were needed and generated the simplest schema that could hold them.

Choices a Human Would Not Have Made

- Language swap from TypeScript to Python without explicit justification or user request.

- SHA-256 for password hashing instead of bcrypt or argon2—a security anti-pattern that any experienced developer would flag immediately.
- A monolithic 1,681-line App.tsx frontend component instead of a modular component hierarchy with routing—the original has dedicated pages, shared components, and React Router v7.
- Omitting the FleetGraph agent entirely. The agent rebuilt the "body" of Ship (CRUD, UI) but skipped its "brain" (proactive monitoring, anomaly detection, on-demand intelligence).
- Using SQLite in production Docker containers with a volume mount instead of PostgreSQL, forgoing concurrent write safety and the original's migration-heavy schema evolution.

3. Performance Benchmarks

Lines of Code

Metric	Original	Rebuilt
Backend LOC	~49,772 (TypeScript)	~2,887 (Python)
Frontend LOC	~47,425 (TypeScript/React)	~2,595 (TypeScript/React)
Total Application LOC	~126,193	~5,482
Ratio (Original : Rebuilt)		23 : 1
E2E Test LOC	~26,324 (Playwright)	~275 (Playwright + Vitest)

The 23:1 LOC ratio reflects both genuine simplification (Python is more concise than TypeScript for CRUD operations, FastAPI auto-generates OpenAPI specs) and significant feature omission. The original includes a 44KB rich text editor component, a 32KB real-time collaboration server, a 60KB issue list with filtering/bulk actions, and a full FleetGraph agent—none of which exist in the rebuild.

Test Coverage

Metric	Original	Rebuilt
Unit Tests	27 (FleetGraph nodes)	35 (backend endpoints)
E2E Tests	73+ (Playwright)	~5 (Playwright smoke)
API Route Tests	13+ test files	Included in unit tests
Coverage Estimate	High (all major flows)	Moderate (CRUD paths only)

Database Complexity

Metric	Original	Rebuilt
Migrations	39	2
Core Tables	10+ (documents, associations, comments, history, workspaces, users, memberships, findings, poll_state, chat)	7 (projects, users, documents, issues, plans, tasks + alembic)
JSONB/Flexible Schema	Yes (properties, content)	No (fixed columns only)
Audit Logging	document_history table	None

Development Speed

Metric	Original	Rebuilt
Development Period	6+ days (85+ commits in 72h window alone)	~24 hours (18 commits, Mar 25-26)
Commits	85+ (in tracked 72h window)	18
Features Delivered	Full PM suite + FleetGraph agent + real-time collab	CRUD dashboard + analytics + Docker

4. Shortcomings

The following catalog documents where the Shipyard agent failed, produced incorrect output, or required intervention during the Ship rebuild. Each entry identifies the limitation it reveals about autonomous coding agents.

FleetGraph Agent Omitted Entirely. The agent never attempted to build the FleetGraph intelligence layer—the proactive polling scheduler, 4-way parallel detection graph (stale issues, missing standups, scope creep, missing rituals), on-demand chat, or HITL approval gates. This is the most architecturally complex part of the original and the agent had no strategy for replicating AI-agent-within-an-application patterns.

Limitation revealed: Autonomous agents struggle with meta-tasks: building another agent system requires understanding the design rationale, not just the API surface. Without explicit instructions to "build a polling scheduler that runs detection nodes in parallel," the agent defaulted to what it could infer from the entity model alone.

Insecure Password Hashing (SHA-256). The agent used hashlib.sha256 for password storage instead of bcrypt, argon2, or passlib. SHA-256 is a fast hash—unsuitable for passwords because it enables brute-force attacks. The original uses proper session-based auth with configurable providers (OIDC, SAML, PIV).

Limitation revealed: The agent optimized for "working code" over "secure code." It chose the simplest hash function that made tests pass. This reveals that without explicit security requirements in the prompt or injected context, agents will choose convenience over correctness for non-functional requirements.

Monolithic Frontend (1,681-line App.tsx). The entire frontend—authentication, project list, user management, document CRUD, issue tracking, plan management, task dashboard, analytics, and project management views—lives in a single App.tsx file. The original decomposes into dozens of focused components with shared design primitives (Radix UI), state management (React Query), and routing (React Router v7).

Limitation revealed: The agent builds files, not architectures. It added features incrementally to a single file because each commit needed to "work," and splitting components mid-stream would require understanding import graphs and shared state. File editing agents gravitate toward fewer, larger files because each edit is a single operation.

No Real-Time Collaborative Editing. The original's TipTap + Yjs integration provides multi-cursor, conflict-free document editing across users. The rebuild has a basic WebSocket channel for broadcasting task events but no document collaboration.

Limitation revealed: CRDT-based collaboration requires deep library integration (Yjs protocol, WebSocket transport, TipTap extension API) that cannot be inferred from endpoint shapes. The agent would need explicit documentation or examples injected as context to attempt this.

Missing Audit Trail and Permission Model. The original tracks every document change in a document_history table, enforces workspace isolation via memberships, supports role-based access (admin/member), and includes CSRF protection. The rebuild has authentication (JWT) but no authorization beyond "logged in or not"—no workspace scoping, no role differentiation, no change history.

Limitation revealed: Permission models are "invisible requirements"—they don't show up in entity schemas or API response shapes. An agent reading the original codebase would see middleware and decorators but may not understand they represent a security boundary rather than boilerplate.

SQLite Instead of PostgreSQL. The agent chose SQLite for the rebuild despite the original running PostgreSQL with features like JSONB columns, expression indexes, and trigger functions (e.g., circular parent prevention). SQLite cannot replicate these.

Limitation revealed: The agent selected the path of least resistance: SQLite requires no external service, no connection pooling configuration, and works with Alembic out of the box. This is a reasonable development choice but a production liability for concurrent writes.

No Section 508 / Accessibility Compliance. The original includes WCAG 2.1 AA compliance with keyboard navigation, screen reader support, proper ARIA attributes, and color contrast validation (tested by 73+ E2E Playwright tests). The rebuild has no accessibility considerations.

Limitation revealed: Accessibility is a cross-cutting concern that requires deliberate attention to every UI component. Autonomous agents generate functional HTML but do not consider screen reader compatibility or keyboard-only navigation unless explicitly prompted.

Simplified Database Schema Lost Extensibility. The original's unified document model with JSONB properties columns allows adding new document types and custom fields without schema migrations. The rebuild's fixed-column tables require a migration for every new field.

Limitation revealed: The agent generated the simplest schema that matched the observed entity types. It had no way to reason about schema extensibility or anticipate future requirements—it optimized for the current set of fields, not for evolution.

Interventions Required During Rebuild. While the rebuild completed in 18 commits, several patterns suggest interventions were needed: commit b76b7c1 ("fix: complete test setup with proper fixture configuration — all 8 tests passing") implies initial test setup failed and required correction; commit 3914148 ("Stabilize backend container startup and migrations") indicates Docker startup issues that needed debugging; commit 0931ae7 ("Harden runtime settings") and f7a93f4 ("Harden production runtime configuration") suggest the agent's initial configuration was insufficient for deployment.

Limitation revealed: Agents produce code that works in the immediate test loop but often fails at integration boundaries (Docker startup, migration ordering, environment variable resolution). These are the seams where autonomous agents most need human oversight.

5. Advances

Despite the shortcomings, the Shipyard agent demonstrated several areas where autonomous development outperformed or matched manual development speed.

Extraordinary Speed: Full Application in ~24 Hours. The agent produced a working, containerized, tested application with 10 API routers, 7 database tables, JWT authentication, Kanban/Gantt views, analytics dashboards, WebSocket real-time updates, and a React frontend—all in approximately 24 hours across 18 commits. A human developer working solo would need 1–2 weeks for equivalent functionality, even with a simpler stack. The agent's throughput on boilerplate CRUD generation is unmatched.

Consistent Code Quality for Repetitive Patterns. Every router follows the same pattern: Pydantic schema validation, SQLAlchemy query, HTTPException for errors, dependency injection for auth. Every model has consistent column definitions, indexes, and relationships. This uniformity is difficult for human developers to maintain across a rapid build—fatigue introduces inconsistencies. The agent's output is mechanically consistent.

Type Safety Throughout. The rebuild uses Python 3.10+ union syntax (int | None), Pydantic models with Field constraints (min_length, max_length, ge, le), model validators for cross-field rules (start_date <= due_date), and Literal types for enums. The TypeScript frontend has full type definitions. This level of type safety was maintained automatically across every file the agent produced.

Docker-Native from Commit One. By commit beac830, the agent had Docker Compose orchestration with separate backend and frontend containers, volume-mounted persistence, nginx proxy configuration, and auto-migration on startup. Containerization is often an afterthought in manual development; the agent treated it as a first-class deliverable.

Test-Driven Progression. The agent wrote 35 test functions covering authentication, CRUD operations, analytics aggregation, WebSocket authentication, date validation, and configuration parsing. Tests were committed incrementally alongside features—not bolted on at the end. The test infrastructure (conftest.py with proper fixtures, test client setup) was established early.

Feature Completeness for the Domain Scope Attempted. Within the project-management domain the agent chose to implement, coverage is thorough: projects have Kanban boards with

four columns (todo, in_progress, in_review, done), Gantt chart data with timeline calculations, assignment workload distribution, and analytics with trend data, status distributions, priority breakdowns, and overdue task detection. These are production-quality features, not stubs.

CI/CD Pipeline Generation. Commit b9ea6a6 added a GitHub Actions CI pipeline. The agent understood that a deployable application needs automated builds and tests—a concern many developers deprioritize during rapid prototyping.

6. Trade-off Analysis

This section evaluates the major architecture decisions made by the Shipyard agent during the rebuild, whether each was the right call given the constraints, and what should change.

LangGraph for Agent Orchestration

Context: LangGraph provides a declarative state machine with automatic state persistence, conditional routing, and native LangSmith tracing. It handles the agent loop, tool dispatch, and message management with minimal custom code.

Verdict: Right call. LangGraph eliminated 500+ lines of custom loop management that would have been needed with a raw API approach. The StateGraph abstraction maps cleanly to the agent's call_llm/execute_tools/should_continue flow. The main cost is framework coupling—but the benefit in development speed and observability outweighs it for a project of this scope.

What would change: No change needed. LangGraph is the right framework for this agent pattern.

Anchor-Based Surgical Editing (Exact String Match)

Context: The agent uses exact-match old_string replacement with triple validation (read-first, staleness check, uniqueness enforcement). This is the same strategy used by Claude Code and aider.

Verdict: Right call with caveats. This strategy works well for targeted edits but degrades on large files where the LLM must reproduce long spans of text exactly. The fuzzy-match hint and line-number feedback for failed edits are well-designed error recovery. The main limitation is that the agent cannot do structural edits (e.g., "move this method to another class") without generating the full old_string for both locations.

What would change: Would add: AST-aware diffing for refactoring operations, and a "batch edit" mode that applies multiple replacements atomically.

Supervisor Multi-Agent Pattern (Coder + Researcher)

Context: The supervisor LLM dispatches to a coder (full tools) or researcher (read-only tools) based on the instruction. Workers run to completion and return summaries.

Verdict: Partially right. The two-worker model covers the common case (investigate, then implement) but lacks parallelism—the supervisor cannot run both workers simultaneously. The context isolation (workers don't see each other's tool calls) is a good design for token efficiency

but means the coder cannot learn from the researcher's intermediate findings, only the summary.

What would change: Would add: parallel worker execution, a shared scratchpad for intermediate findings, and more specialized workers (e.g., a test writer, a reviewer).

Context Compaction at 50% of Window

Context: When conversation history approaches 50% of the 200K context window, older messages are summarized by the LLM and replaced with a compact summary. The last 10 messages are preserved.

Verdict: Aggressive but necessary. 50% is conservative—it triggers compaction earlier than needed, potentially losing useful context. But it provides a buffer against sudden token spikes from large tool outputs. The orphaned-ToolMessage fix (commit 13229e5) shows this was a real failure mode: compaction could remove an AIMessage with tool_calls but leave the ToolMessages, confusing the LLM.

What would change: Would raise threshold to 65–70% and implement selective compaction (summarize tool outputs separately from conversation flow).

Python/FastAPI for the Rebuild (Language Swap)

Context: The agent rebuilt Ship in Python/FastAPI instead of preserving the original TypeScript/Express stack.

Verdict: Wrong call. This decision was not made deliberately—it reflects the agent's training bias toward Python. For a rebuild intended to be compared against the original, preserving the language would have enabled line-by-line comparison, shared test infrastructure, and higher architectural fidelity. The Python rebuild is functionally equivalent for CRUD but cannot be meaningfully compared for code complexity or maintainability because the languages differ.

What would change: Would inject explicit context: "Rebuild this application using the same language and framework as the original." This is a failure of prompt design, not agent capability.

Circuit Breaker (5 Consecutive Error Batches)

Context: The agent halts after 5 consecutive batches where ALL tools fail. Individual tool errors do not trigger the breaker—only complete batch failures.

Verdict: Right call after iteration. The initial implementation (commit ef956b4 fix) triggered on any single tool error, causing premature halts when one tool in a batch of three failed. The corrected version (all-tools-fail threshold) is well-calibrated: it catches genuine dead ends without aborting on recoverable errors.

What would change: Would add: error classification (transient vs. permanent) to route different recovery strategies.

Passive Response Filter (Banned Phrase Detection)

Context: The agent detects when the LLM produces passive language ("let me know," "please confirm") and escalates with system overrides forcing tool calls. Up to 3 retries.

Verdict: Necessary but blunt. This solves a real problem—LLMs frequently default to suggesting actions instead of taking them. The escalation ladder (helpful hint, warning, direct command) is well-structured. However, phrase-matching is fragile: the agent might flag legitimate responses (e.g., "I found the issue—the ensure_ascii flag needs to change") because they contain trigger words. Commit 0ab5cec ("Catch dash-bulleted bold lists") shows ongoing tuning was needed.

What would change: Would replace phrase matching with a classifier or structured output mode that separates "action taken" from "explanation given."

7. If You Built It Again

The following changes would meaningfully improve the agent's rebuild output and the agent itself if the project were started over.

Agent Architecture Changes

Explicit Architecture Extraction Phase. Before writing any code, the agent should run a dedicated "architecture extraction" pass: read the original codebase, identify the tech stack, map the data model, catalog the API surface, and produce a structured plan. The current agent jumps straight to implementation because its system prompt emphasizes action over planning. Adding a mandatory planning phase (even 2–3 LLM calls to produce a schema and endpoint inventory) would have prevented the language swap, the schema simplification, and the FleetGraph omission. The plan should be stored as injected context so it persists across the full rebuild session.

Specialized Workers Beyond Coder/Researcher. The two-worker model is too coarse. A rebuild task benefits from: (1) an Architect worker that reads the original and produces structural requirements, (2) a Coder that implements, (3) a Tester that writes and runs tests, and (4) a Reviewer that compares output against the original. The reviewer would have caught the SHA-256 password issue, the monolithic frontend, and the missing audit trail. Each worker should have a scoped prompt (e.g., the Reviewer's prompt includes "compare against the original codebase at path X").

Parallel Worker Execution. The current supervisor runs workers sequentially. For a rebuild, the researcher could analyze the original codebase while the coder sets up the project skeleton simultaneously. LangGraph supports parallel fan-out natively (the original FleetGraph uses it for its 4-way detection). Adding this would reduce wall-clock rebuild time significantly.

File Editing Strategy Changes

Batch Edit Mode. Many rebuild operations require coordinated edits across multiple files (e.g., adding a new model requires changes to models/__init__.py, a new schema file, a new router, and main.py router registration). Currently each edit is independent; a batch mode that groups related edits and rolls back all if any fail would improve reliability. The agent's rollback_file tool exists but is per-file—transactional rollback across a changeset does not.

File Creation Templates. The rebuild created ~50 Python files with repetitive structure (router pattern, schema pattern, model pattern). Rather than generating each from scratch, the agent

should recognize "this looks like the last 5 files I created" and use the first successful file as a template. This would reduce LLM calls by 40–60% for pattern-heavy tasks and eliminate the inconsistencies that required hardening commits.

AST-Aware Editing for Refactoring. String-based replacement works for targeted changes but cannot safely perform structural operations like "move this function to a new module" or "rename this class across all imports." Adding tree-sitter or ast-based edit operations for Python and TypeScript would enable refactoring without the LLM needing to reproduce exact source text.

Context Management Changes

Original Codebase as Persistent Reference Context. The most impactful change would be injecting the original codebase's structure as permanent context: the database schema, the API endpoint inventory, the component hierarchy, and the FleetGraph architecture. Currently, the agent must "discover" the original through read_file and search_files calls, which is token-expensive and incomplete. A structured summary of the original (generated in the architecture extraction phase) should be injected as context and maintained throughout the rebuild.

Progressive Context Specialization. During the rebuild, different phases need different context. The schema design phase needs the original's data model; the router implementation phase needs the API surface; the frontend phase needs the component hierarchy. Rather than maintaining all context at all times, the agent should swap injected context based on the current task phase. This would keep token usage low while maintaining high-fidelity reference information.

Compaction with Semantic Anchors. The current compaction summarizes old messages generically. For a rebuild task, compaction should preserve structural decisions (e.g., "decided to use SQLAlchemy for ORM," "created 7 tables") as pinned context even when the conversation that produced them is summarized. These decisions are load-bearing—losing them causes the agent to make contradictory choices later.

Process Changes

Checkpoint Validation Against Original. After each major phase (schema, backend, frontend, deployment), the agent should pause and compare its output against the original: "Does my schema cover all document types? Does my API expose the same endpoints? Does my frontend

render the same views?" This human-in-the-loop checkpoint would catch drift early. The PRD's requirement to "document every intervention" implies this was expected but not implemented in the agent's workflow.

Security Requirements as Injected Context. The SHA-256 password issue and missing CSRF/rate-limiting would have been prevented by injecting a security requirements checklist as context: "Passwords must use bcrypt or argon2. All state-changing endpoints must have CSRF protection. Rate limiting must be configured for authentication endpoints." Non-functional requirements must be explicit because agents optimize for functional correctness by default.